UNIVERSIDAD DE GRANADA

Universidad de Granada
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Grado en Ingeniería Informática

Dirección y Gestión de Proyectos (DGP)
Curso Académico 2026–2027

MANUAL DE COORDINACIÓN
Y PLAN DE DIRECCIÓN DEL PROYECTO

Proyecto AUTOVIDA: Sistema de Agenda y Asignación de Tareas Accesible para Alumnado NEAE / PTVAL

En colaboración con el C.E.E. Fundación Purísima Concepción de Granada

Equipo de Trabajo:
CLENCH Software Development

Asignación de Roles:

Miguel Ángel Luque	Coordinador
Yeray Rodríguez Navas	Gestor de Calidad
Julián Carrión Tovar	Gestor Accesibilidad
Pablo Hernández Ibáñez	Moderador
Pablo de la Torre Roldán	Catalogador
José Rodríguez Fernández	Presentador
Amine Azzammouri	Coordinador (Refuerzo)

Índice

1. Imagen de Empresa **1**
1.1. Identidad Corporativa: CLENCH Software Development 1

2. Metodología de Desarrollo y Ciclo de Vida **1**
2.1. Modelo Metodológico Ágil 1
2.2. Ciclo de Vida por Iteraciones 1

3. Recursos Software de Desarrollo **2**
3.1. Ecosistema y Herramientas de Desarrollo 2

4. Organización del Equipo de Trabajo (Estructura, Normas) **3**
4.1. Asignación Nominal de Roles Iniciales 3
4.2. Responsabilidades de Cada Rol 3
4.3. Esquema de Rotación y Protocolo de Ausencias 4
4.4. Matriz de Rotación de Roles por Iteración 5
4.5. Normas de Convivencia y Compromiso Operativo 5

5. Herramientas para Comunicaciones en el Equipo y Reuniones **6**
5.1. Canales Oficiales del Equipo 6
5.2. Protocolo de Reuniones y Elaboración de Actas 6

6. Relaciones con el Cliente (Entrevistas, Reuniones, Revisiones) **6**
6.1. Dinámica de Colaboración Semanal y Entrevistas 6
6.2. Interlocutores y Ética en el Trato 6

7. Estándares de Documentación **7**
7.1. Documentación Textual en LaTeX a través de GitHub y Estructura Normalizada . . 7
7.2. Documentación de Código: Doxygen 7
7.3. Diseño y Modelado Software: UML con Visual Paradigm 7

8. Estándares de Código **7**
8.1. Convenciones de Nomenclatura y Lenguaje 8
8.2. Comentarios y Documentación en Código 8
8.3. Principios de Calidad y Formato 8

9. Plan de Gestión de Cambios **8**
9.1. Procedimiento de Evaluación y Aprobación 8

10. Control de Versiones (Método y Herramientas) **8**
10.1. Estrategia y Disciplina en el Repositorio 9
10.2. Convención de Commits (Conventional Commits) 9

11. Gestión de Calidad (Durante el Desarrollo y al Final, Herramientas) **9**
11.1. Procedimiento de Calidad en Cada Iteración 9
11.2. Proceso de Hardening y Verificación Global 10
11.3. Cuadro Oficial de Herramientas de Calidad 10

12. Plan de Medición del Desempeño, Recompensas y Castigos **10**
12.1. Control del trabajo individual 10
12.2. Sistema de Recompensas y Reconocimiento 10
12.3. Régimen Sancionador Interno (Castigos y Penalizaciones) 10

1. Imagen de Empresa

1.1. Identidad Corporativa: CLENCH Software Development

El equipo de trabajo opera bajo la identidad **CLENCH Software Development**. Esta iniciativa combina el aprendizaje con el desarrollo de software de calidad para promover la responsabilidad social y la inclusión de personas en situación de vulnerabilidad.



- **Denominación Oficial:** CLENCH Software Development.
- **Lema y Propósito:** «*Tecnología accesible para potenciar la autonomía personal*».
- **Misión:** Diseñar, implementar y validar soluciones digitales accesibles,que faciliten la rutina diaria de estudiantes con Necesidades Específicas de Apoyo Educativo (NEAE).
- **Valores:**
  1. **Accesibilidad y Empatía:** La persona usuaria (estudiantes PTVAL y profesorado de la Fundación Purísima Concepción) es el núcleo de cada decisión de diseño.
  2. **Calidad:** Compromiso con el código limpio, análisis estático, cobertura de pruebas y estabilidad.
  3. **Transparencia y Coordinación:** Comunicación fluida, reparto equitativo de esfuerzo y sinceridad en la estimación de plazos y riesgos.

2. Metodología de Desarrollo y Ciclo de Vida

2.1. Modelo Metodológico Ágil

Para desarrollar la aplicación utilizaremos **metodología ágil**, integrando los eventos de trabajo de Scrum para la gestión del flujo de trabajo continuo y la monitorización de tareas en curso (*Work In Progress*). Este esquema equilibra la flexibilidad en la asimilación de feedback de usuarios reales con el control formal de alcance, costes, riesgos y plazos de DGP.

2.2. Ciclo de Vida por Iteraciones

El proyecto se ejecuta bajo un modelo de **prototipado evolutivo iterativo**, con 3 iteraciones y una verificación final:

1. **Fase Inicial:** Organizamos el equipo y redactamos este Manual de Coordinación. También preparamos la propuesta técnica con la arquitectura y el presupuesto inicial, definimos los perfiles de usuario y escenarios de uso, y dejamos listos tanto la pila de producto (Product Backlog) como el plan de entregas.
2. **Iteración 1:** En esta etapa nos centramos en los primeros pasos del desarrollo. Comenzamos levantando los servicios que necesitaremos luego, como la base de datos, el entorno del backend y el entorno del frontend. Implementamos el inicio de sesión adaptado y la gestión de perfiles

accesibles para sacar nuestro primer prototipo funcional y poder probarlo. Tras esta iteración tendremos la capacidad de personalizar la interfaz de la aplicación según las necesidades de cada alumno.

3. **Iteración 2:** Nos centramos en desarrollar el sistema de tareas. Tras esta iteración debemos poder iniciar sesión y crear, modificar y eliminar tareas genéricas. La relación entre cada profesor y padre con los alumnos debe ser funcional a la hora de que los profesores y padres creen tareas y les aparezcan a los alumnos.

4. **Iteración 3:** Tomando como base el sistema de tareas comenzaremos a crear las plantillas de tareas necesarias para actividades del centro: los pasos guiados para encargar fotocopias (reprografía), la gestión de los alumnos que no comen en el comedor, la guía de visitas y un temporizador visual adaptado.Añadimos la parte de rutinas personales configurables. Además, aprovechamos para pulir los colores, contrastes y tamaños de los botones, dejando toda la aplicación estable.

5. **Hardening y Verificación Global (Cierre de Iteración 3):** Antes de dar el proyecto por cerrado, hacemos una validación completa con el cliente, ponemos la aplicación al límite para detectar posibles fallos y revisamos que todo cumpla con la protección de datos (RGPD) y la seguridad básica.

6. **Defensa Final (12 de enero de 2027):** Exponemos el proyecto y hacemos una demostración en vivo de cómo funciona la aplicación ante los profesores y los representantes del colegio C.E.E. Purísima Concepción.

3. Recursos Software de Desarrollo

La aplicación se desarrollará en **Flutter**. Trabajaremos principalmente con el **IDE Android Studio** con las extensiones de Dart y Flutter, por las facilidades que nos brinda a la hora de hacer tests unitarios y probar los prototipos en los propios dispositivos físicos.

3.1. Ecosistema y Herramientas de Desarrollo

- **Framework de Desarrollo Frontend: Flutter SDK (Dart)**. Permite generar aplicaciones nativas multiplataforma desde un único código fuente, asegurando un rendimiento fluido en las tabletas.
- **Entorno de Desarrollo (IDE de Flutter): Visual Studio Code / Android Studio** con las extensiones oficiales de Flutter y Dart. Ofrece:
  - *Hot Reload* y *Hot Restart* para modificar y personalizar componentes gráficos y estilos visuales en tiempo de ejecución de manera instantánea.
  - Suite integrada para ejecución automatizada y depuración de pruebas unitarias (*Flutter Test*).
  - *Flutter DevTools* para análisis de rendimiento, inspección del árbol de widgets y perfiles de memoria.
- **Backend y API REST: Python con FastAPI**, aportando tipado seguro, arquitectura desacoplada y alta escalabilidad en la gestión de servicios web.
- **Persistencia y Base de Datos: PostgreSQL** para almacenar y gestionar los datos de la aplicación.

- **Catálogo de Pictogramas:** Catálogo oficial de pictogramas de **ARASAAC** (licencia Creative Commons) para comunicación aumentativa.
- **Diseño UX/UI: Figma** para la confección y testeo de wireframes accesibles previos a la fase de programación.

4. Organización del Equipo de Trabajo (Estructura, Normas)

4.1. Asignación Nominal de Roles Iniciales

Se seguirá una división de roles entre los 7 integrantes del equipo, rotando aquellos roles menos importantes en cada iteración para repartir dicha carga. La asignación inicial de roles del equipo es:

- **Pablo Hernández Ibáñez:** Moderador (Rotativo)
- **Yeray Rodríguez Navas:** Gestor de Calidad (**Fijo**)
- **Pablo de la Torre Roldán:** Catalogador (Rotativo)
- **José Rodríguez Fernández:** Presentador (Rotativo)
- **Julián Carrión Tovar:** Gestor de Usabilidad y Accesibilidad (**Fijo**)
- **Miguel Ángel Luque:** Coordinador (Rotativo)
- **Amine Azzammouri:** Coordinador (Rotativo inverso) (Refuerzo)

4.2. Responsabilidades de Cada Rol

Cada rol tendrá las siguientes responsabilidades:

- **Coordinador:** Organiza el trabajo del equipo, distribuye tareas, controla los plazos y realiza el seguimiento general del proyecto. Se encarga de avisar sobre las próximas entregas y de planificar el tiempo de trabajo correspondiente a cada iteración. Actúa como portavoz de gestión ante el profesorado y supervisa el equilibrio de la dedicación horaria.
- **Catalogador:** Responsable de recopilar, analizar y clasificar la información generada por el grupo en las distintas tareas asociadas a las prácticas. Se encarga de revisar y consolidar la información añadida a la documentación en cada iteración y, cuando sea posible, de revisar la documentación de la iteración anterior en coordinación con el Gestor de Calidad. Asimismo, custodia y organiza la documentación en el repositorio institucional (`docs/`) y coordina la publicación de actas de reunión.
- **Moderador:** Responsable de plantear y moderar los debates, así como de seleccionar las ideas y decisiones grupales. En caso de indecisión, tiene la última palabra tras consultar con el Gestor de Calidad sobre la opción más adecuada. Asimismo, se encarga de gestionar y documentar los cambios en los documentos de especificación.
- **Presentador:** Se encarga de realizar todas las comunicaciones con el cliente, así como de elaborar presentaciones que muestren el avance y los resultados del proyecto. Prepara los resúmenes que se comunicarán al cliente o profesor y decide, junto con los roles pertinentes, qué contenidos y preguntas se expondrán en cada iteración.

- **Gestor de la Calidad (Rol Fijo):** Responsable de asegurar la calidad de los productos generados y del proceso de desarrollo. Vela por que la documentación y las técnicas aplicadas cumplan con los estándares definidos:
  - Asegurarse de que se usan herramientas estandarizadas para especificación y diseño (diagramas UML en Visual Paradigm y plantillas de ingeniería de software).
  - Asegurarse de que se organiza bien el trabajo: celebración regular de reuniones, toma de actas formales, reparto equitativo de tareas y uso de recursos hardware y software correctos.
  - Revisar o asegurarse de que se revisen y prueben todos los entregables (verificando correctitud y completitud) y de que se finalizan a tiempo o se planifica adecuadamente cualquier cambio de fecha y sus consecuencias.
  - Supervisar los criterios de aceptación, revisiones de código, análisis estático continuo y auditorías periódicas de calidad del proceso y producto.
- **Gestor de Usabilidad y Accesibilidad (Rol Fijo):** Responsable de planificar, coordinar y velar por la usabilidad y la accesibilidad del proyecto. Se encarga de investigar e informar al equipo sobre las buenas prácticas, técnicas y herramientas a utilizar en estas materias. Asimismo, define los requisitos de accesibilidad, formula las preguntas necesarias de las consultas que se harán y verifica su correcto cumplimiento mediante pruebas de usuario, testing o entrevistas. Para el desarrollo e implementación de estas actividades contará con el apoyo de los roles correspondientes:
  - Formarse y formar a sus compañeros en guías de usabilidad y accesibilidad web y móvil (WCAG 2.1 AA, heurísticas de accesibilidad cognitiva y diseño accesible).
  - Planificar qué guías considerar en función de los usuarios finales del proyecto (alumnado PTVAL con NEAE, docentes y familias).
  - Formarse en heurísticas de accesibilidad y emplearlas rigurosamente para evaluar y probar la aplicación.
  - Aplicar lectores de pantalla (**TalkBack** en Android y **VoiceOver** en iOS) para validar la accesibilidad móvil.
  - Aplicar validadores de accesibilidad (**Lighthouse**, **WAVE**, Google Accessibility Scanner) para evaluar contrastes cromáticos, etiquetas y comprensión.
  - Elaborar informes periódicos del progreso del trabajo en cuanto a usabilidad y accesibilidad.
- **Refuerzo (Rol Rotativo Inverso):** Replica las funciones y responsabilidades de uno de los roles rotativos para apoyar a su responsable, compartir la carga de trabajo y garantizar la continuidad de sus tareas. En cada iteración, el rol que se refuerza cambia siguiendo el orden inverso de la rotación general (*ver siguiente apartado*) La persona de refuerzo colaborará con el titular del rol, sin sustituir su responsabilidad sobre las decisiones y entregables correspondientes. A pesar de esta especificación, pueden surgir cambios en función de la necesidad de cada iteración.

4.3. Esquema de Rotación y Protocolo de Ausencias

Los roles de **Gestor de Calidad** y **Gestor de Usabilidad y Accesibilidad** serán **fijos** a los responsables correspondientes (Yeray Rodríguez Navas y Julián Carrión Tovar), mientras que el resto rotará en cada iteración de trabajo.

Regla de Rotación Cíclica

La secuencia obligatoria de rotación en cada cambio de iteración es:
$\ldots \longrightarrow$ **Coordinador** $\longrightarrow$ **Catalogador** $\longrightarrow$ **Moderador** $\longrightarrow$ **Presentador** $\longrightarrow \ldots$

Regla de Rotación Cíclica Inversa para Rol de Refuerzo

La secuencia de rotación del rol de refuerzo en cada cambio de iteración es:
$\ldots \longrightarrow$ **Coordinador** $\longrightarrow$ **Presentador** $\longrightarrow$ **Moderador** $\longrightarrow$ **Catalogador** $\longrightarrow \ldots$

Protocolo de Suplencias y Ausencias:

1. En caso de que algún integrante con cualquiera de los roles de Gestor (Calidad o Accesibilidad) no pueda acudir a una iteración de trabajo, dicho rol será cedido temporalmente al que en dicha reiteración tenga el rol de **Coordinador** (y al **Presentador** en caso de inasistencia de ambos), el rol de **Refuerzo** se encargará de cubrir el trabajo del rol que ha tomado el relevo del gestor faltante.

2. Si algún rol rotativo no está presente en una iteración, se le cederá durante esa iteración al responsable del rol de **Refuerzo**, en caso de falta de dicho rol, la responsabilidad se cedera al miembro que será el encargado de dicho rol en la siguiente iteración.

4.4. Matriz de Rotación de Roles por Iteración

Integrante	Fase Inicial	Iteración 1	Iteración 2	Iteración 3
Miguel Ángel Luque	Coordinador	Catalogador	Moderador	Presentador
Pablo de la Torre Roldán	Catalogador	Moderador	Presentador	Coordinador
Pablo Hernández Ibáñez	Moderador	Presentador	Coordinador	Catalogador
José Rodríguez Fernández	Presentador	Coordinador	Catalogador	Moderador
Yeray Rodríguez Navas	Gestor Calidad	Gestor Calidad	Gestor Calidad	Gestor Calidad
Julián Carrión Tovar	Gestor Accesib.	Gestor Accesib.	Gestor Accesib.	Gestor Accesib.
Amine Azzammouri	Refuerzo: Coordinador	Refuerzo: Presentador	Refuerzo: Moderador	Refuerzo: Catalogador

4.5. Normas de Convivencia y Compromiso Operativo

1. **Puntualidad:** Se establece un margen máximo de cortesía de 20 minutos para las reuniones presenciales y virtuales, salvo que exista una justificación previa.

2. **Notificación Previa de Bloqueos:** Cualquier impedimento, ausencia o retraso previsible deberá notificarse con al menos **24 horas de antelación**. En circunstancias imprevistas debidamente justificadas, se aceptará una notificación realizada fuera de dicho plazo.

3. **Registro Honesto de Horas:** Cada integrante deberá cumplimentar semanalmente su ficha individual de dedicación horaria, indicando de forma aproximada y veraz las horas empleadas.

5. Herramientas para Comunicaciones en el Equipo y Reuniones

5.1. Canales Oficiales del Equipo

- **WhatsApp:** Principalmente utilizaremos WhatsApp para la organización y concreción de fechas y horarios, así como para la difusión de avisos inmediatos o urgencias imprevistas.
- **Discord:** Plataforma destinada a realizar videollamadas cuando no podamos hacer reuniones presenciales, compartir pantalla en sesiones técnicas y vertebrar discusiones temáticas.
- **Jira:** Utilizaremos la herramienta Jira para el seguimiento y control de las tareas en progreso, gestión del Product Backlog, diagramas de Gantt y órdenes del día de reuniones.
- **Correo Institucional (@correo.ugr.es y @go.ugr.es):** Canal oficial reservado para la comunicación formal con el profesorado y el personal directivo del centro.

5.2. Protocolo de Reuniones y Elaboración de Actas

- **Reuniones Semanales Internas:** Encuentro semanal del equipo (presencial o por Discord) convocado formalmente mediante Jira o WhatsApp.
- **Actas Formales:** Toda reunión oficial se documentará obligatoriamente según la plantilla estandarizada (`docs/plantillas/plantilla_acta_reunion.md`). El Catalogador y el Moderador redactarán el acta y la publicarán en `docs/reuniones/` con plazo aproximado para antes de la siguiente reunión.

6. Relaciones con el Cliente (Entrevistas, Reuniones, Revisiones)

6.1. Dinámica de Colaboración Semanal y Entrevistas

Se realizará una visita al colegio para obtener la información extra necesaria no aportada en la entrevista con el profesorado. La información adicional que necesitemos se obtendrá mediante el correo de contacto proporcionado por el cliente y las reuniones con el profesorado. El objetivo primordial será comprobar la evolución del proyecto, validar las funcionalidades desarrolladas y resolver posibles dudas.

También se propone utilizar las sesiones semanales de prácticas como reuniones con el cliente (el profesor de prácticas de la asignatura) para revisar el avance del proyecto y validar funcionalidades.

Los cambios y decisiones acordados en estas reuniones se registrarán metódicamente para mantener actualizada la planificación y los requisitos del proyecto en todo momento.

6.2. Interlocutores y Ética en el Trato

- **Entorno de Aplicación:** Personal directivo, docentes y profesionales del **C.E.E. Fundación Purísima Concepción de Granada**.
- **Destinatarios Finales:** Alumnado del Programa de Transición a la Vida Adulta (PTVAL), tutores escolares y familias.
- **Protección de Datos y RGPD:** Se observará máxima cautela y cumplimiento de las leyes de protección de datos de menores y personas con discapacidad, garantizando anonimato en datos y fotografías.

7. Estándares de Documentación

7.1. Documentación Textual en LaTeX a través de GitHub y Estructura Normalizada

Para la documentación textual y técnica, es decir, especificaciones, documentos de desarrollo, diseño y manuales que requieran redacción formal, utilizaremos LaTeX gestionado de forma centralizada y versionado a través de **GitHub**, siendo que a su vez se realizará la redacción de la docuemtanción en la plataforma **Prism** para facilitar el desarrollo compartido de la documentación. Esta elección asegura un riguroso control de versiones mediante Git, homogeneidad tipográfica corporativa y generación determinista de documentos en formato PDF de alta calidad, facilitando de la misma manera el trabajo paralelo sin conflicto de versiones gracias a Prism.

Para asegurar que todos los entregables compartan una imagen impecable y profesional, **la estructura que seguirá cada documento será obligatoriamente la siguiente:**

1. **Portada:** Logotipo oficial de CLENCH Software Development, escudo de la Universidad de Granada, título formal del documento, subtítulo del proyecto AUTOVIDA, asignaturas (DGP/MDA), fecha y listado completo de autores con roles asignados.
2. **Índice:** Estructura paginada y jerarquizada generada automáticamente con numeración decimal.
3. **Introducción y Objetivos del Documento:** Propósito, justificación y alcance del documento.
4. **Cuerpo Central de Contenido:** Secciones y subsecciones numeradas de forma jerárquica, tablas con formato corporativo, figuras legibles y recuadros de aviso.
5. **Conclusiones y Siguientes Pasos** (Si fuese necesario)
6. **Referencias y Anexos:** Fuentes bibliográficas, enlaces a repositorios y glosarios.

Versionado: El código fuente `.tex` y todos los recursos asociados (figuras, imágenes y tablas) se mantendrán y versionarán en el repositorio Git de GitHub bajo la ruta `docs/`. Las modificaciones documentales se gestionarán mediante el flujo estándar de ramas y revisiones mediante *Pull Requests*, asegurando la aprobación del equipo. Los documentos finales se compilarán en formato PDF y se dispondrá, si se considera necesario, adicionalmente de versiones en Markdown cuando sea conveniente para su consulta rápida en la propia plataforma de GitHub.

7.2. Documentación de Código: Doxygen

Con respecto a la documentación de código, hemos optado por usar **Doxygen**, aplicando así un estándar sencillo y fácil de entender para el grupo y cualquiera que pueda heredar el código, además de permitir la generación de documentación de código de manera dinámica a partir de los propios comentarios en los ficheros fuente.

7.3. Diseño y Modelado Software: UML con Visual Paradigm

Para la concreción del diseño de nuestra solución software (base de datos, definición de clases, diagramas de arquitectura y casos de uso) utilizaremos el estándar **UML**. Para ello nos apoyaremos en plataformas diseñadas para ello, como **Visual Paradigm** entre otras.

8. Estándares de Código

Utilizaremos la herramienta **Flutter** para la implementación. Se aplican las siguientes reglas de estilo y diseño:

8.1. Convenciones de Nomenclatura y Lenguaje

- **Idioma de Nomenclatura:** Los nombres de las variables, funciones, métodos, clases y archivos estarán escritos en inglés para mayor estandarización.
- **Variables y Funciones:** Se utilizará la forma lower-case / `lowerCamelCase` (ejemplo: `taskStatus`, `studentId`, `calculateTotalTime()`, `fetchActiveUsers()`).
- **Clases y Widgets:** Se empleará `PascalCase` de acuerdo con el estándar de Flutter (ejemplo: `TaskDetailCard`, `VisualTimerWidget`, `UserRepository`).
- **Constantes y Enumerados:** Se utilizará `UPPER_SNAKE_CASE` o `lowerCamelCase` según la convención oficial de Dart (`const int MAX_RETRY = 3;`).

8.2. Comentarios y Documentación en Código

Los comentarios de los scripts, funciones y demás estructuras se pondrán al inicio del documento o del respectivo bloque de código para mayor legibilidad y orden, empleando etiquetas y bloques compatibles con Doxygen (`///` o `/** ... */`). Describirán propósito, parámetros, retornos y excepciones controladas.

8.3. Principios de Calidad y Formato

- Código modular, desacoplado y siguiendo principios SOLID y Clean Code.
- Formateo automático mandatorio mediante `dart format .` antes de cada commit.

9. Plan de Gestión de Cambios

Ante cualquier solicitud de **refactorización del código**, se revisarán los requisitos afectados y se evaluará el impacto del cambio en la planificación, el diseño y el desarrollo de la aplicación. Una vez aprobado, se realizarán las modificaciones necesarias tanto en la aplicación como en la documentación y la memoria del proyecto, manteniendo los requisitos debidamente actualizados. Este procedimiento podrá simplificarse cuando el cambio sea insignificante, siempre que se comunique previamente a través de los canales de comunicación del equipo de trabajo.

9.1. Procedimiento de Evaluación y Aprobación

1. **Notificación del Cambio:** La persona solicitante registrará el cambio en Jira, indicando su descripción, justificación, prioridad y requisitos afectados.
2. **Análisis de Impacto:** Los coordinadores y el Gestor de Calidad evaluarán la repercusión sobre el cronograma en Jira, el esfuerzo en horas de los miembros, los costes y la accesibilidad.
3. **Decisión Formal:** Si el cambio es aprobado por el Coordinador, el Gestor de Calidad y el Moderador, se actualizan los diagramas UML y memorias.

10. Control de Versiones (Método y Herramientas)

Utilizaremos la herramienta **GitHub** para el control de versiones, actualizando el repositorio compartido en cada sesión de prácticas, llevando a cabo un control supervisado y documentado correctamente, especificando qué cambios se han llevado a cabo.

10.1. Estrategia y Disciplina en el Repositorio

- **Actualización Sistemática:** Es obligatorio realizar commit y push de los avances generados al término de cada sesión presencial de prácticas, documentando de forma el trabajo efectuado.
- **Control Supervisado y Pull Requests:** Las ramas funcionales de trabajo deberán ser integradas mediante Pull Requests revisadas antes de su incorporación a la rama principal de desarrollo para evitar conflictos.
- **Prohibición de Código Roto:** Ningún cambio se integrará en la rama principal sin haber sido revisado por el equipo, especialmente por el **Gestor de Calidad**. Cuando existan pruebas unitarias, deberá comprobarse mediante `flutter test` que todas se ejecutan satisfactoriamente sobre el código modificado.

10.2. Convención de Commits (Conventional Commits)

Formato obligatorio: `<tipo>: <descripción concisa>`

- `feat`: Nueva funcionalidad implementada.
- `fix`: Corrección de un defecto o fallo.
- `docs`: Actualización exclusiva de documentación o actas.
- `style`: Cambios estéticos o de formateo sin impacto lógico.
- `refactor`: Modificación de código sin alteración funcional.
- `test`: Incorporación o modificación de pruebas unitarias/integración.
- `a11y`: Modificaciones específicas de accesibilidad (contrastes, VoiceOver/TalkBack).
- `chore`: Mantenimiento de configuración, dependencias o CI/CD.

11. Gestión de Calidad (Durante el Desarrollo y al Final, Herramientas)

Se revisará completamente todo el desarrollo al final de cada iteración y se corregirán los aspectos necesarios para alcanzar nuestros objetivos, asegurando una detección temprana de errores y validación continua del trabajo entregado.

11.1. Procedimiento de Calidad en Cada Iteración

Por cada iteración seguiremos rigurosamente los siguientes pasos:

1. **Definición de criterios de aceptación:** Discutiremos sobre lo que se ha de realizar en la iteración y qué criterios seguimos para determinar si se ha completado correctamente (Definition of Ready y Definition of Done).
2. **Revisiones de código:** Se revisará el código obligatoriamente en busca de errores antes de añadirlo a la rama de desarrollo de la nueva iteración. Se usarán herramientas de debug e IA para logar la revisión temprana.
3. **Análisis Estático Continuo:** Escaneo automatizado del código fuente en cada ciclo para identificar vulnerabilidades, duplicidades y fallos de estilo.

4. **Pruebas Unitarias y de Integración:** Ejecución de pruebas automatizadas sobre los nuevos módulos desarrollados en la iteración correspondiente. Este paso puede llegar a ser omitido en las primeras iteraciones debido a la falta de contenido para la realización de pruebas.

5. **Demostración y Validación de la Iteración:** Al finalizar cada iteración, se realizará una revisión funcional con el cliente para presentar y validar el trabajo desarrollado. Esta revisión permitirá comprobar que los resultados obtenidos satisfacen sus necesidades y expectativas.

11.2. Proceso de Hardening y Verificación Global

Al terminar la tercera iteración, se llevará a cabo el **proceso de hardening y verificación global**, el cual constará de:

- Una revisión funcional integral por parte del cliente.
- Una auditoría final exhaustiva que compruebe el correcto funcionamiento de la aplicacion.

11.3. Cuadro Oficial de Herramientas de Calidad

Fase / Ámbito	**Herramienta**	Aplicación en el Proyecto
Control de Versiones & CI/CD	**GitHub**	Gestión del repositorio, control de Pull Requests y ramas.
Análisis Estático & Estilo	**SonarCloud / ESLint**	Detección automática de bugs, vulnerabilidades y formato de código.
Pruebas Unitarias & API	**Jest**	Automatización de pruebas de lógica e integración de servicios.
Accesibilidad & UX	**Lighthouse / WAVE**	Evaluación de niveles de accesibilidad (contraste, etiquetas, lectura).
Gestión de Tareas y Bugs	**Jira**	Trazabilidad de requisitos, incidencias y feedback de las demos.

12. Plan de Medición del Desempeño, Recompensas y Castigos

12.1. Control del trabajo individual

Cada miembro registrará sus horas de trabajo de manera aproximada en el documento (`docs/plantillas/plantilla_registro_horas.md`).

12.2. Sistema de Recompensas y Reconocimiento

- **Reconocimiento interno:** Con el registro de las horas de trabajo de cada miembro, se valorará que la carga de los integrantes sea similar.
- **Al final de cada iteración:** Se premiará el trabajo realizado (siempre que cumpla con los estánderes de calidad del gestor de calidad) con una tarde de tapas.

12.3. Régimen Sancionador Interno (Castigos y Penalizaciones)

Para asegurar la responsabilidad individual y salvaguardar los resultados del grupo frente a actitudes negligentes:

1. **Inasistencia a Reuniones:** La falta injustificada de un sujeto a una reunión conllevará su registro en el informe individual del mismo. Si ocurren varias faltas injustificadas se notificará al docente responsable de la case de prácticas.
2. **Incumplimiento de Entregables o Plazos:** El retraso o no entrega de tareas comprometidas sin previo aviso justificado de al menos **48 horas** será comunicado al docente responsable de las prácticas de la asignatura.

Documento consensuado y aprobado por el Equipo CLENCH Software Development.
Grado en Ingeniería Informática — Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación.
Universidad de Granada, Curso Académico 2026–2027.